

Main chapter ko section-by-section simple Hinglish mein explain karunga, saath hi confusing pairs jaise JavaScript vs ECMAScript, transpiler vs polyfill, aur interpreted vs compiled ko examples se clear karunga. Figures actual input mein available nahi hain, isliye unka concept text ke basis par samjhaunga. # Chapter 1: What Is JavaScript?

Easy aur Detailed Hinglish Explanation

Is chapter ka main purpose JavaScript ka syntax sikhana nahi hai. Iska purpose yeh samjhana hai ki **JavaScript asal mein kya hai, ise kaun manage karta hai, browser aur JavaScript mein kya difference hai, naye features purane browsers mein kaise chalte hain, aur JavaScript code execute kaise hota hai.**

1. JavaScript seekhna ek journey hai

Author kehta hai:

JavaScript ko koi bhi completely nahi jaanta.

JavaScript kaafi broad aur continuously evolving language hai. Aap kitna bhi seekh lo, hamesha koi naya concept ya kisi purane concept ki deeper understanding mil sakti hai.

Isliye JavaScript learning ko destination ki tarah mat dekho:

1

 Maine JavaScript complete kar li.

Iske badle ise direction ki tarah dekho:

1

 Main JavaScript ko pehle se better samajh raha hoon.

Is book ko kaise padhna chahiye?

Is book ko jaldi finish karne ki koshish nahi karni chahiye. Important cheezein:

- patience rakho
- concepts ko properly process karo
- examples khud likho
- code execute karke dekho
- sirf reading par depend mat raho

Kya yeh beginner book hai?

Book ka naam **Get Started** hai, lekin yeh complete beginners ke liye basic introduction book nahi hai.

Author assume karta hai ki:

- aapko JavaScript ka basic syntax pata hai

- aap variables, functions, arrays aur objects dekh chuke ho
- aapne kuch mahine JavaScript code likha hai

Is book ka goal aapko JavaScript ki deeper study ke liye prepare karna hai.

JavaScript ke teen main pillars

Aage series mein JavaScript ko mainly teen foundational pillars ke through samjhaya jayega:

1. **Scope aur Closures**
2. **Prototypes aur Objects**
3. **Types aur Coercion**

JavaScript mein bahut saare features hain, lekin language ki deeper understanding largely in teen topics par depend karti hai.

2. JavaScript naam kahan se aaya?

JavaScript ka naam bahut confusing hai, kyunki isme **Java** word aata hai.

Log aksar sochte hain:

- JavaScript, Java ka simplified version hai
- JavaScript sirf Java ke scripts likhne ke liye hai
- JavaScript aur Java closely related hain

Lekin yeh assumptions galat hain.

JavaScript ka original naam

Jab Brendan Eich ne language create ki thi, tab:

1. iska code name **Mocha** tha
2. Netscape ke andar ise **LiveScript** kaha gaya
3. public release ke time naam **JavaScript** rakh diya gaya

JavaScript naam kyun rakha gaya?

Us waqt Java kaafi popular tha. JavaScript ko Java programmers ke liye familiar aur attractive dikhane ke liye yeh naam use kiya gaya.

Yeh mainly ek **marketing decision** tha, technical relationship nahi.

JavaScript ko theoretically kuch aur bhi kaha ja sakta tha, jaise:

Java aur JavaScript similar kyun lagte hain?

Dono languages mein kuch syntax similar dikhta hai:

```
1  if (condition) {  
2      // code  
3  }
```

Dono mein:

- blocks ke liye { }
- statement ke end par ;
- conditions ke liye if
- loops ke liye for

use hote hain.

Lekin iska reason yeh nahi ki JavaScript, Java se nikli hai. Reason yeh hai ki dono languages par **C aur C++ style syntax ka influence** hai.

Simple analogy:

Java aur JavaScript ka relation utna hi hai jitna ham aur hamster ka.

Naam similar hai, lekin cheezein alag hain.

3. JavaScript, JS aur ECMAScript

Aapko teen terms frequently milengi:

- JavaScript
- JS
- ECMAScript

JavaScript aur JS

JS, JavaScript ka commonly used short form hai.

Book mein bhi mostly JavaScript ki jagah JS use kiya gaya hai.

ECMAScript kya hai?

JavaScript language ka official standard **ECMAScript** kehlata hai.

ECMAScript specification define karti hai:

- language ka syntax
- variables ka behavior

- functions ka behavior
- objects ka behavior
- operators ka behavior
- errors kab aayengi
- language ke built-in features

Simple way:

- | | |
|---|--|
| 1 | ECMAScript = JavaScript ke official rules |
| 2 | JavaScript engine = un rules ka implementation |

Browser aur Node.js mein chalne wala JavaScript, ECMAScript standard ka implementation hota hai.

ES2019 ka kya matlab hai?

2016 ke baad ECMAScript versions ko generally year ke basis par identify kiya jaane laga:

- ES2016
- ES2017
- ES2018
- ES2019

Chapter January 2020 ke context mein likha gaya tha, isliye isme **ES2019** ko current standard bataya gaya hai.

Ye historical context hai. Iska matlab yeh nahi ki ES2019 aaj ka newest JavaScript standard hai.

“JS6” aur “ES8” kyun avoid karna chahiye?

Book recommend karti hai ki aise terms avoid karo:

- | | |
|---|-----|
| 1 | JS6 |
| 2 | ES8 |

Prefer karo:

- | | |
|---|--------|
| 1 | JS |
| 2 | ES2019 |
| 3 | ES2020 |

Ya general modern JavaScript ke liye simply:

- | | |
|---|------------|
| 1 | JavaScript |
|---|------------|

4. JavaScript language ko kaun manage karta hai?

JavaScript ki language specification ko mainly **TC39** manage karti hai.

TC39 kya hai?

TC39 ek technical committee hai jo JavaScript language ke future par kaam karti hai.

Committee mein different companies ke representatives participate karte hain, jaise:

- browser makers
- device makers
- web platform companies

Committee ke members:

- naye features propose karte hain
- existing behavior discuss karte hain
- proposals review karte hain
- changes par vote karte hain
- final changes ECMA ko submit karte hain

ECMA phir un changes ko official ECMAScript standard ka part banata hai.

Proposal stages

Naye JavaScript features ek five-stage process se guzarte hain:

- | | |
|---|---------|
| 1 | Stage 0 |
| 2 | Stage 1 |
| 3 | Stage 2 |
| 4 | Stage 3 |
| 5 | Stage 4 |

Programming mein counting often zero se shuru hoti hai, isliye stages bhi zero-based hain.

Stage 0

Kisi TC39 member ko lagta hai ki idea explore karne layak hai.

Is stage par feature:

- sirf initial idea ho sakta hai
- change ho sakta hai
- reject ho sakta hai
- production ke liye reliable nahi hota

Kisi random blog ya social media post par proposed feature automatically Stage 0 nahi hota. TC39 ke kisi member ko us idea ko officially champion karna hota hai.

Stage 1 se Stage 3

Proposal gradually define, review aur test hota hai.

Jaise-jaise stage badhti hai:

- proposal zyada mature hota hai
- syntax aur behavior clearer hota hai
- implementations aur testing improve hoti hai

Stage 4

Stage 4 ka matlab proposal complete aur standard mein include hone ke liye ready hai.

Aisa feature next yearly ECMAScript revision mein add ho sakta hai.

Proposal ko complete hone mein kitna time lagta hai?

Ek proposal ko Stage 0 se Stage 4 tak pahunchne mein:

- kuch months
- ya several years

lag sakte hain.

Isliye kisi early proposal ko dekh kar turant production application mein use nahi karna chahiye.

5. JavaScript ki multiple versions nahi hain

Ek common misconception hai:

Chrome ka JavaScript alag hai, Firefox ka JavaScript alag hai.

Chapter ke according modern web mein officially **ek hi JavaScript language** hai, jiske rules ECMAScript specification define karti hai.

Different browsers mein different JavaScript engines hote hain:

- Chrome mein V8
- Firefox mein SpiderMonkey

Engines alag ho sakte hain, lekin unka target same specification ko implement karna hai.

Phir browser differences kyun aate hain?

Possible reasons:

1. Ek browser ne naya feature jaldi implement kar diya

2. Dusre browser ne abhi implement nahi kiya
3. Browser-provided Web APIs ka behavior different hai
4. Purane web compatibility issues hain
5. JavaScript engine ya browser mein bug hai

Important difference:

1 Different implementation timing ≠ different JavaScript language

JScript kya tha?

Early 2000s mein Microsoft ka ek JavaScript-like implementation tha jise **JScript** kaha jaata tha.

Us waqt genuinely compatibility differences the. Lekin modern JavaScript ecosystem mein woh situation largely historical hai.

Isliye aaj yeh kehna ki internet par multiple completely different JavaScript languages chal rahi hain, inaccurate hoga.

6. Web JavaScript ke decisions ko influence karta hai

JavaScript kai environments mein chal sakta hai:

- browsers
- Node.js servers
- robots
- smart devices
- embedded systems

Lekin JavaScript ki history aur evolution mein web ka influence sabse zyada hai.

“We don’t break the web”

Agar specification mein koi change technically better ho, lekin us change se millions of existing websites toot sakti hain, toh browsers us change ko implement karne se mana kar sakte hain.

Aise case mein TC39 ko practical web reality consider karni padti hai.

contains() se includes()

Arrays ke liye pehle ek method ka naam contains() plan kiya gaya tha.

Lekin pata chala ki kuch existing websites aur old frameworks mein contains naam ka use aise ho raha tha ki new standard method se websites break ho sakti thi.

Isliye method ka naam change karke:

```
1 array.includes(value);
```

kar diya gaya.

flatten() se flat()

Array flattening method ke liye pehle flatten() naam consider hua. Existing web compatibility issue ke karan final method bana:

```
1 array.flat();
```

Is situation ko community mein “smooshgate” ke naam se bhi discuss kiya gaya.

Main lesson:

JavaScript sirf technically ideal language design ke according evolve nahi hoti. Existing web content ki compatibility bhi bahut important hai.

7. Appendix B kya hai?

ECMAScript specification ka **Appendix B** web browsers ke liye kuch additional aur historical features document karta hai.

Kuch old features core modern JavaScript design ka ideal part nahi hain, lekin browsers unhe remove nahi kar sakte, kyunki old websites un par depend kar sakti hain.

Examples:

- 0 prefix wale old octal literals
- escape()
- unescape()
- old string methods
- old RegExp methods

Is features ko mainly historical browser compatibility ke liye rakha gaya hai.

Kya inhe new code mein use karna chahiye?

Generally nahi.

Aapko:

- standard modern syntax use karna chahiye
- browser-specific historical behavior par depend nahi karna chahiye
- deprecated constructs avoid karne chahiye

Appendix B ko exception list ki tarah samjho, recommended feature list ki tarah nahi.

8. Har JavaScript-looking feature JavaScript ka part nahi hai

Consider this:

```
1 alert("Hello, JS!");
```

Yeh valid JavaScript code hai, lekin alert() function ECMAScript language specification ka part nahi hai.

alert() kahan se aata hai?

alert() browser environment provide karta hai.

JavaScript specification function call ka syntax define karti hai:

```
1 someFunction();
```

Lekin alert function ka actual behavior browser define karta hai.

JavaScript aur environment APIs

Different environments JavaScript ko different capabilities provide karte hain.

Browser provides

```
1 alert();
2 fetch();
3 getUserMedia();
```

Node.js provides

```
1 fs.write();
```

Common environments provide

```
1 console.log();
```

console.log() bhi core ECMAScript language ka official feature nahi hai. Yeh host environment provide karta hai.

Guest analogy

Book ke according:

```
1 alert() call JavaScript syntax hai
2 alert function browser ka guest hai
```

JavaScript rules decide karte hain function ko call kaise karna hai. Browser decide karta hai alert box kaise dikhana hai.

Browser differences ka actual source

Log bolte hain:

JavaScript browsers mein inconsistent hai.

Lekin kai baar inconsistency JavaScript language mein nahi, balki browser APIs mein hoti hai.

Distinction yaad rakho:

1	JavaScript language
2	vs
3	Browser-provided APIs

9. Developer Console pure JavaScript environment nahi hai

Browser DevTools console ya Node REPL mein code execute karna convenient hota hai:

1	<code>let x = 10;</code>
2	<code>x + 5;</code>

Console turant output dikha deta hai.

Lekin console ka purpose exact program-file behavior simulate karna nahi hai. Console ka purpose developer ko quick experimentation ki facility dena hai.

REPL ka meaning

REPL ka full form hai:

1	Read
2	Evaluate
3	Print
4	Loop

Console:

1. input read karta hai
2. input evaluate karta hai
3. result print karta hai
4. process repeat karta hai

Console aur .js file mein differences

Differences in areas mein ho sakte hain:

- global var declaration
- global let aur const
- repeated declarations
- strict mode
- top-level this
- hoisting across separate entries
- browser global object

Example:

```
1 let name = "Ruth";
```

Aap console mein same declaration dobara run karo:

```
1 let name = "Alex";
```

Console ka behavior ek actual .js file ya separate script ke behavior se different ho sakta hai.

Important lesson

Console experimentation ke liye useful hai, lekin exact language semantics test karne ka perfect source nahi hai.

Use console for:

- quick calculations
- method testing
- small experiments
- debugging
- variable inspection

Actual program behavior test karne ke liye:

- .js file banao
- proper module use karo
- actual runtime environment mein run karo

10. JavaScript multi-paradigm language hai

Programming paradigm ka matlab hai code ko organize aur problems solve karne ka broad style.

Teen common paradigms hain:

1. Procedural
 2. Object-oriented
 3. Functional
-

10.1 Procedural programming

Procedural style mein code step-by-step operations ke form mein organize hota hai.

```
1  const price = 100;
2  const tax = price * 0.18;
3  const total = price + tax;
4
5  console.log(total);
```

Flow top-to-bottom hai:

1. price define karo
 2. tax calculate karo
 3. total calculate karo
 4. result display karo
-

10.2 Object-oriented programming

Object-oriented style mein related data aur behavior ko objects ya classes ke andar group kiya jaata hai.

```
1  class Product {
2      constructor(price) {
3          this.price = price;
4      }
5
6      getTotal() {
7          return this.price + this.price * 0.18;
8      }
9  }
10
11  const product = new Product(100);
12  console.log(product.getTotal());
```

Yahan:

- data: price
- behavior: getTotal()

ek Product class ke andar organize hain.

10.3 Functional programming

Functional style mein logic functions ke through organize hota hai. Functions ko values ki tarah use kiya ja sakta hai.

```
1  const addTax = price => price + price * 0.18;
2
3  console.log(addTax(100));
```

Functional programming generally emphasize karti hai:

- small functions
 - predictable output
 - data transformations
 - functions ko values ki tarah use karna
 - unnecessary mutation avoid karna
-

JavaScript ki flexibility

Kuch languages ek specific paradigm ko strongly prefer karti hain:

- C mainly procedural
- Java heavily class-oriented
- Haskell strongly functional

JavaScript mein aap teeno styles use kar sakte ho:

```
1  // procedural
2  let total = 100;
3
4  // object-oriented
5  const product = {
6      price: 100
7  };
8
9  // functional
10 const double = value => value * 2;
```

Aap same project mein paradigms mix bhi kar sakte ho.

Isliye JavaScript ko **multi-paradigm language** kaha jaata hai.

Iska matlab yeh nahi ki har jagah teeno styles mix karna zaroori hai. Aapko problem aur project ke according suitable approach choose karni chahiye.

11. Backwards compatibility kya hai?

Backwards compatibility ka meaning:

Jo code pehle valid JavaScript tha, future JavaScript update use normally invalid nahi banayega.

Example ke liye, 1995 mein likha valid JavaScript ideally aaj ke engine mein bhi run karna chahiye.

Iske benefits

Developers confidence ke saath applications bana sakte hain:

```
1 Browser ke next update ke baad mera valid code randomly invalid nahi hoga.
```

Websites kai years tak exist karti hain. Agar har language update old code tod de, toh web maintain karna bahut difficult ho jayega.

Iska cost kya hai?

Backwards compatibility ka matlab mistakes ko remove karna difficult hai.

Agar koi poorly designed feature language ka official part ban gaya aur websites uska use karne lagi, toh TC39 use easily remove nahi kar sakti.

Kyunki:

```
1 Feature remove kiya
2 → old website break hui
3 → users affected hue
```

Isliye naye features language mein add karne se pehle bahut carefully evaluate kiye jaate hain.

Kya kabhi breaking changes nahi hote?

Bahut rare cases mein backwards-incompatible change ho sakta hai.

Lekin usse pehle:

- existing websites ka usage study hota hai
- impact estimate hota hai
- browser makers discuss karte hain

- benefit aur risk compare hote hain

Aise changes usually rare corner cases mein hote hain.

12. Forwards compatibility kya hai?

Forwards compatibility ka meaning:

Nayi language feature use karne wala code purane engine mein bhi bina break hue chal sake.

JavaScript forwards-compatible nahi hai.

Example:

```
1  const user = {  
2      profile: {  
3          name: "Ruth"  
4      }  
5  };  
6  
7  console.log(user?.profile?.name);
```

Optional chaining ?. ko jo old engine samajhta hi nahi, wahan syntax error aa sakta hai.

Backwards vs forwards compatibility

Backwards compatibility

```
1  Old code → new engine mein chale
```

JavaScript largely backwards-compatible hai.

Forwards compatibility

```
1  New code → old engine mein chale
```

JavaScript forwards-compatible nahi hai.

13. HTML aur CSS ka behavior different kyun hai?

HTML aur CSS unknown features ko often ignore kar sakte hain.

Agar old browser kisi CSS property ko nahi samajhta:

```
1  .some-element {
```

```
2     old-known-property: value;
3     new-unknown-property: value;
4 }
```

Toh browser unknown declaration skip kar sakta hai aur baaki CSS continue process kar sakta hai.

Programming language mein aisa safely karna difficult hai.

Consider:

```
1  const payment = calculatePayment();
2  sendReceipt(payment);
```

Agar engine pehli unknown statement skip kar de, toh next line ke paas valid payment hi nahi hoga.

Programming statements ek dusre par depend karte hain. Isliye JavaScript engine arbitrary unknown syntax ko ignore karke safely continue nahi kar sakta.

14. New JavaScript aur old browsers ke beech gap

Suppose aap modern JavaScript feature use kar rahe ho, lekin user ka browser purana hai.

Do types ke gaps ho sakte hain:

Syntax gap

Old engine nayi syntax ko parse nahi kar pata.

Example:

```
1  let x = 3;
```

Bahut old engine let ko understand nahi karega. Result:

```
1  Syntax error
```

Program execution start hone se pehle fail ho sakta hai.

API gap

Engine syntax samajhta hai, lekin method available nahi hai.

Example:

```
1  Object.is(a, b);
```

Agar Object.is() old environment mein available nahi hai, toh program us line tak chalega, phir runtime error dega.

In dono problems ke solutions alag hain:

- 1 New syntax → transpilation
- 2 Missing API → polyfill

15. Transpilation kya hai?

Transpilation mein modern source code ko equivalent older syntax mein convert kiya jaata hai.

Popular transpiler:

- 1 Babel

Original modern code

```
1  if (something) {  
2      let x = 3;  
3      console.log(x);  
4  }  
5  else {  
6      let x = 4;  
7      console.log(x);  
8  }
```

let block-scoped hota hai. Dono blocks ka x alag hai.

Transpiler old-compatible form generate kar sakta hai:

```
1  var x$0, x$1;  
2  
3  if (something) {  
4      x$0 = 3;  
5      console.log(x$0);  
6  }  
7  else {  
8      x$1 = 4;  
9      console.log(x$1);  
10 }
```

Yahan variables ke unique names use karke similar behavior preserve kiya gaya.

Transpiler ki zarurat kyun?

Aap soch sakte ho:

Direct old syntax kyun na likhein?

Reason yeh hai ki modern syntax often:

- clearer hoti hai
- intent better communicate karti hai
- safer hoti hai
- maintain karna easier hota hai

Recommended workflow:

```
1 Developer modern JavaScript likhe
2           ↓
3 Babel transpile kare
4           ↓
5 Old-compatible JavaScript generate ho
6           ↓
7 Browser generated code run kare
```

Transpilation target fixed nahi hota

Aap kin browsers ko support karna chahte ho, uske according transpilation hoti hai.

Jaise hi project old browser support drop karta hai, transpilation target newer ho sakta hai.

Isliye target ek **sliding window** hai.

16. Polyfill kya hai?

Polyfill missing API ka JavaScript-based replacement hota hai.

Suppose modern environment mein method available hai:

```
1 promise.finally(callback);
```

Lekin old environment mein `Promise.prototype.finally` exist nahi karta.

Polyfill check karega:

```
1 if (!Promise.prototype.finally) {
2   Promise.prototype.finally = function (callback) {
3     // replacement behavior
4   };
5 }
```

Check kyun kiya jaata hai?

Condition ensure karti hai ki:

- old environment mein missing method add ho
- new environment ka native method overwrite na ho

Flow:

```
1 Method already available?  
2   Yes → native method use karo  
3   No  → polyfill define karo
```

Transpiler aur polyfill mein difference

Transpiler

Syntax convert karta hai.

```
1 const add = (a, b) => a + b;
```

Possible output:

```
1 var add = function (a, b) {  
2   return a + b;  
3 };
```

Polyfill

Missing method ya API provide karta hai.

```
1 if (!Array.prototype.includes) {  
2   // includes ka replacement  
3 }
```

Easy rule:

```
1 Syntax problem = transpiler  
2 Feature/API missing = polyfill
```

Book wale polyfill ko directly use kyun nahi karna?

Chapter mein diya Promise.prototype.finally polyfill sirf concept explain karne ke liye hai.

Woh completely specification-compliant nahi hai.

Production code mein:

- trusted polyfill library use karo
 - officially maintained implementation use karo
 - random hand-written polyfill par depend mat karo
-

17. JavaScript interpreted hai ya compiled?

Yeh chapter ka important aur slightly technical section hai.

Common statement:

JavaScript interpreted language hai.

Author ke according reality isse zyada complex hai. JavaScript execution se pehle parse aur process hoti hai, isliye behavior ke perspective se use compiled language ke closer samajhna useful hai.

Traditional interpretation model

Traditional interpreted model ko roughly aise samjho:

1	Line 1 execute
2	Line 2 execute
3	Line 3 execute
4	Line 4 execute
5	Line 5 par error discover

Agar line 5 par syntax problem hai, toh line 1 se 4 tak ka code already run ho chuka ho sakta hai.

Parsed/compiled model

Compiled-style processing:

1	Poora code parse karo
2	↓
3	Errors check karo
4	↓
5	Executable form generate karo
6	↓
7	Program execute karo

Agar parsing ke time line 5 par invalid syntax mil gayi, toh execution start hi nahi hota.

18. Parsing aur AST kya hai?

JavaScript engine source code ko directly human-readable lines ki tarah blindly execute nahi karta.

Pehle engine code ko parse karta hai.

Example:

```
1  const total = price + tax;
```

Parser is code ki structure identify karta hai:

- variable declaration hai
- variable ka naam total hai
- right side par addition expression hai
- addition mein price aur tax operands hain

Is structured representation ko generally **Abstract Syntax Tree**, ya AST, kaha jaata hai.

Conceptually:

```
1  VariableDeclaration
2  └─ name: total
3  └─ value: AdditionExpression
4      └─ price
5      └─ tax
```

Yeh actual engine output ka exact format nahi, bas concept samjhane ke liye simplified representation hai.

19. Early errors kya hoti hain?

JavaScript specification kuch errors ko execution se pehle detect karne ki requirement deti hai.

Inhe early errors kaha jaata hai.

Example:

```
1  function example(a, a) {
2      "use strict";
3  }
```

Strict mode mein duplicate parameter names early error cause kar sakte hain.

Engine ko yeh error execution se pehle identify karna hota hai. Iske liye code ko pehle parse karna zaroori hai.

Isse prove hota hai ki JavaScript simple top-to-bottom line interpreter ki tarah kaam nahi karti.

20. JavaScript execution ka complete flow

Chapter ke according JavaScript program ka broad flow kuch aisa hota hai:

Step 1: Developer code likhta hai

```
1  const message = "Hello";  
2  console.log(message);
```

Step 2: Build tools code process kar sakte hain

Example tools:

- Babel
- Webpack
- aur doosre build tools

Possible tasks:

- transpilation
- bundling
- optimization
- module combination

Step 3: JavaScript engine code parse karta hai

Source code AST mein convert hota hai.

```
1  Source code → AST
```

Step 4: Intermediate form generate hota hai

Engine AST ko bytecode ya kisi intermediate representation, yani IR, mein convert kar sakta hai.

```
1  AST → Bytecode / IR
```

Step 5: JIT optimization hoti hai

JIT ka full form:

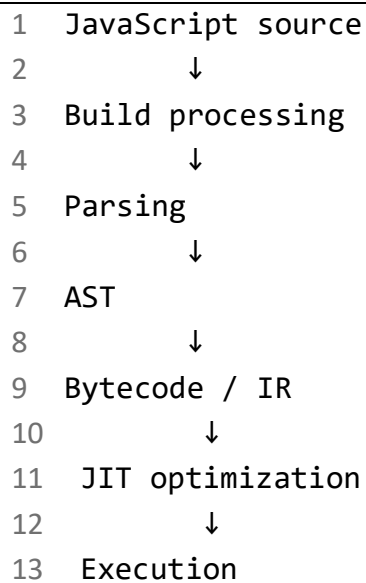
```
1  Just-In-Time
```

Engine runtime ke dauran frequently executed code identify karke optimized machine-level form produce kar sakta hai.

Step 6: JavaScript VM code execute karti hai

Finally JavaScript Virtual Machine generated instructions execute karti hai.

Complete simplified flow:



Main conclusion

Author ka conclusion:

JavaScript spirit aur practical behavior mein compiled language ke closer hai.

Iska important benefit yeh hai ki malformed syntax aur kuch static errors execution se pehle detect ho jaati hain.

21. ASM.js kya tha?

JavaScript performance improve karna language aur browser engines ka major concern raha hai.

2013 mein Mozilla engineers ne Unreal Engine 3 ko C se JavaScript-compatible form mein port karke browser mein high performance par run karke dikhaya.

Is performance mein **ASM.js** style ka role tha.

ASM.js kya hai?

ASM.js regular valid JavaScript ka restricted subset/style tha.

Code unusual style mein likha jaata tha taaki JavaScript engine ko type-related hints mil saken aur engine better optimization kar sake.

Important point:

ASM.js normally humans ke manually likhne ke liye nahi tha.

Expected workflow:

```
1 C/C++ source
2     ↓
3 Tool conversion
4     ↓
5 ASM.js style JavaScript
6     ↓
7 Browser engine
```

22. WebAssembly kya hai?

ASM.js ke baad **WebAssembly**, ya WASM, introduce hua.

WASM ka purpose non-JavaScript languages ke programs ko efficient form mein web platform par run karne ka path provide karna tha.

WASM JavaScript jaisa source code nahi hai

ASM.js valid JavaScript tha. WASM JavaScript syntax nahi hai.

WASM generally compact binary format ke form mein distribute kiya jaata hai.

```
1 C / C++ / Rust jaise source
2     ↓
3 Ahead-of-time compilation
4     ↓
5 WASM binary
6     ↓
7 Runtime execution
```

WASM ka advantage

JavaScript source ke saath engine ko generally:

- parse karna
- AST banana
- compile/process karna

padta hai.

WASM pehle se compact executable-like format mein hota hai, isliye initial processing comparatively efficient ho sakti hai.

Kya WASM sirf performance ke liye hai?

Performance important motivation thi, lekin WASM ka broader goal bhi hai:

- non-JS languages ko web par lana
- different language features ko support karna
- cross-platform execution provide karna
- ek portable runtime target banana

Kya WASM JavaScript ko replace karega?

Chapter ka clear answer:

Nahi, WASM JavaScript ko replace nahi karega.

WASM, JavaScript ko remove karne ke badle web platform ki capabilities augment karta hai.

JavaScript useful rahegi for:

- UI logic
- DOM interaction
- event handling
- application behavior
- browser APIs
- web application coordination

WASM useful ho sakta hai for:

- performance-heavy computations
- games
- media processing
- code originally written in other languages
- portable computation modules

Dono ek saath kaam kar sakte hain.

TypeScript directly WASM ke liye perfect kyun nahi?

WASM heavily static type information par depend karta hai.

TypeScript source mein static types hoti hain, lekin normal TypeScript compilation types ko remove karke JavaScript generate karti hai. Isliye TypeScript automatically ideal WASM source language nahi ban jaati.

AssemblyScript jaise projects JavaScript/TypeScript-like syntax aur WASM ke beech gap bridge karne ki koshish karte hain.

23. Strict mode kya hai?

Strict mode ES5, yani 2009 mein introduce hua.

Iska purpose developers ko safer aur cleaner JavaScript likhne mein help karna tha.

Strict mode ko sirf restriction mat samjho. Ise ek built-in guide ya language-level linter ki tarah samjho.

Strict mode ke benefits

Strict mode:

- problematic patterns ko disallow karta hai
 - silent mistakes ko errors mein convert kar sakta hai
 - code ko predictable banata hai
 - team collaboration improve karta hai
 - engine optimization ke liye better conditions de sakta hai
-

24. File-level strict mode

Entire file ke liye strict mode enable karne ke liye:

```
1 "use strict";  
2  
3 const name = "Ruth";
```

"use strict"; file ke beginning mein hona chahiye.

Isse pehle only:

- whitespace
- comments

allowed hone chahiye.

Example:

```
1 // This file uses strict mode  
2 "use strict";  
3  
4 function greet() {  
5     console.log("Hello");  
6 }
```

Stray semicolon problem

Agar pragma se pehle semicolon aa gaya:

```
1 ;  
2 "use strict";
```

Toh "use strict" strict-mode directive ke roop mein work nahi karega.

JavaScript ke liye "use strict"; ek valid string expression bhi ho sakti hai. Isliye necessarily error nahi aayega, lekin strict mode silently enable nahi hoga.

25. Function-level strict mode

Strict mode ko ek specific function ke andar bhi enable kar sakte hain:

```
1 function someOperations() {  
2     "use strict";  
3  
4     // Function ka code strict mode mein chalega  
5 }
```

Function-level strict mode kab use karein?

Mainly tab jab:

- old non-strict file bahut badi ho
- poori file ko ek saath strict mode mein convert nahi kar sakte
- gradually functions migrate kar rahe ho

New code mein generally entire file ya module-level strict behavior better hai.

26. Strict mode mein kya change hota hai?

Duplicate parameters disallowed ho sakte hain

```
1 "use strict";  
2  
3 function add(a, a) {  
4     return a + a;  
5 }
```

Strict mode mein yeh early error de sakta hai.

Default this behavior

Non-strict function call mein this global object se bind ho sakta hai.

Strict mode mein normal function call ke andar this:

```
1 undefined
```

ho sakta hai.

Example:

```
1 "use strict";
2
3 function checkThis() {
4     console.log(this);
5 }
6
7 checkThis();
```

Yahan this generally undefined hoga.

Yeh accidental global-object usage ko prevent karta hai.

27. Strict mode default kyun nahi bana diya gaya?

Agar browsers suddenly har old script ko strict mode mein treat karne lagte, toh old websites break ho sakti thi.

JavaScript ki backwards compatibility policy ke karan strict mode ko general classic scripts ke liye automatically default banana difficult hai.

Phir modern code mein strict mode common kaise hai?

Do major reasons:

1. Transpiled code

Build tools often generated code ko strict mode mein output karte hain.

2. ES modules

ES6 modules automatically strict mode mein run karte hain.

Example:

```
1 import { something } from "../module.js";
```

Module file ke andar manually yeh likhna necessary nahi:

```
1 "use strict";
```

Module code by default strict hota hai.

Isliye technically classic scripts mein strict mode optional hai, lekin modern production JavaScript mein yeh largely **de facto default** ban gaya hai.

28. Chapter ka final definition of JavaScript

Is chapter ke according JavaScript ko is tarah define kar sakte hain:

1. JavaScript ECMAScript standard ka implementation hai

JavaScript ke core syntax aur behavior ko ECMAScript specification define karti hai.

2. TC39 language ko guide karti hai

TC39 new proposals aur language changes ko manage karti hai. ECMA standard ko formally host aur publish karta hai.

3. JavaScript different environments mein chalti hai

Examples:

- web browsers
- Node.js
- devices
- embedded systems

Har environment extra APIs provide kar sakta hai.

4. JavaScript aur Web APIs same cheez nahi hain

Examples:

```
1 alert();
2 fetch();
3 console.log();
```

JavaScript syntax mein use hote hain, lekin inka implementation environment provide karta hai.

5. JavaScript multi-paradigm hai

Aap code ko likh sakte ho:

- procedural style mein
- object-oriented style mein

- functional style mein
- in styles ke combination mein

6. JavaScript backwards-compatible hai

Old valid JavaScript ko newer engines mein working rakhne ki strong commitment hai.

7. JavaScript forwards-compatible nahi hai

New syntax automatically old engines mein nahi chalegi.

Solution:

- syntax ke liye transpilation
- missing APIs ke liye polyfills

8. JavaScript parsed aur compiled nature rakhti hai

Engine:

1. code parse karta hai
2. AST banata hai
3. intermediate representation generate karta hai
4. JIT optimizations kar sakta hai
5. code execute karta hai

9. Strict mode better JavaScript encourage karta hai

Modern JavaScript modules automatically strict mode mein run karte hain.

10. WASM JavaScript ka replacement nahi hai

WASM web ko new languages aur performance-heavy programs support karne ki additional capability deta hai.

Transpiler aur Polyfill ka final practical example

Suppose aap yeh modern code likhte ho:

```
1  const users = data?.users ?? [];  
2  
3  users.forEach(user => {  
4      console.log(user.name);  
5  });
```

Is code mein modern syntax ho sakti hai:

- const
- optional chaining ?.
- nullish coalescing ??
- arrow function =>

Old browser in syntax forms ko parse nahi kar sakta.

Solution: Transpiler

Babel equivalent older syntax generate kar sakta hai.

Ab suppose aap use karte ho:

```
1 users.includes(currentUser);
```

Agar old browser Array.prototype.includes() provide nahi karta, toh transpiler sirf syntax conversion se method create nahi karega.

Solution: Polyfill

includes() ka polyfill missing API provide karega.

Final distinction:

- | | |
|---|--|
| 1 | Babel/transpiler: |
| 2 | Modern syntax ko old syntax mein convert karta hai. |
| 3 | |
| 4 | Polyfill: |
| 5 | Missing built-in API ka replacement provide karta hai. |

Quick Revision Notes

- | | |
|---|---|
| 1 | JavaScript Java ka version nahi hai. |
| 1 | ECMAScript JavaScript ka official standard hai. |
| 1 | TC39 JavaScript language proposals manage karti hai. |
| 1 | Stage 4 proposal standard mein include hone ke liye ready hota hai. |
| 1 | Modern ecosystem mein official JavaScript language ek hi hai. |
| 1 | alert(), fetch() aur console.log() core JavaScript specification ka part nahi, |
| 2 | environment-provided APIs hain. |
| 1 | Browser console pure .js file jaisa environment nahi hota. |
| 1 | JavaScript procedural, object-oriented aur functional styles support karti hai. |
| 1 | JavaScript backwards-compatible hai, forwards-compatible nahi. |

- | |
|---|
| 1 New syntax ke liye transpiler use hota hai. |
| 2 Missing API ke liye polyfill use hota hai. |
| 1 JavaScript execution se pehle parse aur compile/process hoti hai. |
| 1 WASM JavaScript ko replace nahi karta. |
| 1 ES modules automatically strict mode mein run karte hain. |

Is Chapter ka Sabse Important Message

Agar is poore chapter ko ek idea mein summarize karein:

JavaScript sirf browser mein line-by-line chalne wali chhoti scripting language nahi hai. Yeh standardized, multi-paradigm, backwards-compatible aur compiled nature wali sophisticated programming language hai, jise TC39 aur ECMA ke through carefully evolve kiya jaata hai.

Aur practical developer ke liye message hai:

Modern, clean aur readable JavaScript likho. Old environments ko support karne ke liye transpilers aur polyfills jaise tools use karo. Browser APIs, console behavior aur core JavaScript language ko ek hi cheez mat samjho.

Chapter mein diye gaye external links

- ECMAScript 2019 specification: <https://www.ecma-international.org/ecma-262/10.0/>
- TC39 proposal process: <https://tc39.es/process-document/>
- TC39 proposals repository: <https://github.com/tc39/proposals>
- Babel: <https://babeljs.io>
- ECMAScript 2019 Appendix B: <https://www.ecma-international.org/ecma-262/10.0/#sec-additional-ecmascript-features-for-web-browsers>

Explanation approach

Maine chapter ko original order ke close rakhte hue simpler concepts, practical examples aur clear comparisons mein break kiya hai. Special focus un areas par rakha gaya hai jahan beginners ko confusion hoti hai, especially **JavaScript vs Java**, **JavaScript vs browser APIs**, **backwards vs forwards compatibility**, **transpiler vs polyfill**, aur **interpreted vs compiled**.